

Preparación de Entrevista — Arquitecto de Software

Antonio García Márquez · Proyecto de referencia: **mini-uber / driver-service**

1. Perfil profesional

Ingeniero de software con experiencia en el diseño y desarrollo de sistemas distribuidos basados en microservicios. Enfoque en arquitecturas escalables, resilientes y mantenibles, con especial atención a las buenas prácticas de diseño (SOLID, DDD, arquitectura hexagonal) y a la calidad del código (pruebas automatizadas, integración continua).

Puntos clave a comunicar

- Capacidad para diseñar sistemas desacoplados y orientados a eventos.
- Experiencia práctica llevando servicios desde el diseño hasta producción.
- Visión de arquitecto: equilibrio entre simplicidad, escalabilidad y coste de mantenimiento.
- Buenas prácticas de documentación y comunicación técnica con el equipo.

2. Arquitectura del proyecto de referencia: mini-uber

mini-uber es un sistema tipo "ride-hailing" (similar a Uber) compuesto por microservicios independientes que colaboran entre sí. **driver-service** es el microservicio responsable de la gestión de los conductores (alta, disponibilidad, ubicación, estado del vehículo, etc.) dentro de dicho ecosistema.

Responsabilidades de driver-service

- Registro y gestión del ciclo de vida de los conductores.
- Actualización y consulta del estado/disponibilidad en tiempo real.
- Exposición de una API para que otros servicios (por ejemplo, el servicio de viajes o de asignación) puedan consultar y reservar conductores disponibles.
- Publicación de eventos de dominio (ej. "conductor disponible", "conductor asignado") para mantener desacoplados a los servicios consumidores.

Decisiones arquitectónicas a defender

Aspecto	Decisión	Justificación
Estilo de arquitectura	Microservicios	Escalado independiente, despliegues autónomos y aislamiento de fallos por dominio.
Comunicación	Síncrona (REST/gRPC) para consultas, asíncrona (eventos) para notificaciones	Reduce el acoplamiento temporal entre servicios y mejora la resiliencia ante caídas puntuales.

Persistencia	Base de datos propia por servicio (database per service)	Evita acoplamiento a nivel de esquema y permite evolución independiente del modelo de datos.
Resiliencia	Circuit breaker / timeouts / reintentos en llamadas entre servicios	Evita el efecto cascada de fallos en un sistema distribuido.

3. Puntos fuertes a destacar

- **Arquitectura** Diseño de sistemas distribuidos desacoplados mediante eventos y APIs bien definidas (contract-first).
- **Calidad** Cultura de testing (unitario, integración, contract testing entre servicios) e integración continua.
- **Escalabilidad** Diseño pensado para el crecimiento horizontal: servicios sin estado, cacheo, particionado de datos.
- **Observabilidad** Logging estructurado, métricas y trazabilidad distribuida para depurar problemas entre microservicios.
- **Liderazgo técnico** Capacidad de guiar decisiones de arquitectura y mentorizar a otros desarrolladores.
- **Comunicación** Traducir decisiones técnicas complejas en términos comprensibles para perfiles no técnicos.

4. Preguntas y respuestas típicas de entrevista

4.1 Preguntas técnicas de arquitectura

¿Por qué elegirías una arquitectura de microservicios frente a un monolito?

Los microservicios permiten escalar y desplegar cada componente de forma independiente, aíslan fallos por dominio y facilitan que equipos distintos trabajen en paralelo. A cambio, aumentan la complejidad operativa (observabilidad, consistencia de datos, comunicación entre servicios), por lo que solo se justifican cuando el tamaño del equipo y del producto lo requiere. Un monolito bien modularizado puede ser preferible en etapas tempranas.

¿Cómo garantizas la consistencia de datos entre microservicios como driver-service y el servicio de viajes?

Evito transacciones distribuidas síncronas (2PC) y opto por consistencia eventual mediante el patrón Saga, coordinando el flujo con eventos de dominio (por ejemplo, "DriverAssigned", "TripCancelled") y compensaciones si un paso falla. Esto prioriza la disponibilidad del sistema sobre la consistencia inmediata.

¿Cómo evitarías que un fallo en un servicio downstream tumbe todo el sistema?

Aplicando patrones de resiliencia: timeouts agresivos, circuit breaker (ej. Resilience4j), reintentos con backoff exponencial, bulkheads para aislar recursos y respuestas degradadas (fallback) cuando sea posible, en lugar de propagar el fallo en cascada.

¿Cómo diseñarías la comunicación entre driver-service y el resto de servicios?

Para consultas puntuales de estado (¿está disponible este conductor?) usaría comunicación síncrona vía REST o gRPC con contratos versionados. Para notificar cambios de estado a múltiples consumidores (asignación, facturación, notificaciones push) usaría eventos asíncronos a través de un bus de mensajes (Kafka/RabbitMQ), desacoplando productores y consumidores en el tiempo.

¿Cómo gestionarías el versionado de la API de driver-service?

Versionado explícito (por ejemplo, en la URL o en cabeceras), manteniendo compatibilidad hacia atrás siempre que sea posible, evolucionando los contratos de forma aditiva y comunicando de forma clara las fechas de deprecación de versiones antiguas.

¿Qué estrategia seguirías para la base de datos de un servicio con alta carga de lectura, como la consulta de conductores disponibles?

Separar lecturas y escrituras (CQRS ligero), añadir una capa de caché (Redis) para las consultas más frecuentes, indexar adecuadamente por ubicación/estado y considerar particionado geográfico si el volumen lo requiere.

4.2 Preguntas de comportamiento y experiencia

Cuéntame sobre un momento en el que tuviste que tomar una decisión de arquitectura difícil.

Estructura la respuesta con el método STAR: Situación (contexto del proyecto), Tarea (el problema concreto a resolver), Acción (las alternativas evaluadas y por qué elegiste una) y Resultado (impacto medible: rendimiento, mantenibilidad, reducción de incidencias). Ejemplo: decidir entre acoplar dos servicios vía llamada síncrona o desacoplarlos con eventos, explicando el trade-off de latencia frente a resiliencia.

¿Cómo gestionas el desacuerdo técnico dentro de un equipo?

Fomento la discusión basada en datos y no en opiniones: pruebas de concepto, benchmarks o documentos de diseño (ADR - Architecture Decision Records) que comparen alternativas. Busco el consenso, pero si no se alcanza, se toma una decisión con un responsable claro, documentando el porqué para poder revisarla más adelante si las circunstancias cambian.

Describe una vez que tuviste que aprender una tecnología nueva rápidamente.

Explica tu proceso de aprendizaje (documentación oficial, prototipos pequeños, pair programming) y cómo lo aplicaste de forma segura en el proyecto real, por ejemplo introduciendo la tecnología primero en un componente de bajo riesgo antes de generalizarla.

¿Cómo priorizas entre entregar rápido y mantener la calidad/arquitectura limpia?

Reconozco la deuda técnica como una herramienta válida cuando se gestiona de forma consciente: documento la decisión, su coste futuro y planifico cuándo abordarla. La calidad no es negociable en aspectos críticos (seguridad, integridad de datos), pero sí puede haber margen en aspectos no críticos a corto plazo.

¿Cómo comunicas decisiones de arquitectura a perfiles no técnicos (negocio, producto)?

Traduzco el impacto técnico a términos de negocio: tiempo, coste, riesgo y beneficio. Evito la jerga y uso analogías o diagramas simples, centrándome en qué gana el negocio (fiabilidad, velocidad de entrega futura) en lugar de en el detalle de implementación.

5. Preguntas para hacer al entrevistador

- ¿Cómo está estructurado el equipo de arquitectura y qué grado de autonomía tienen los equipos de producto sobre sus propios microservicios?
- ¿Qué herramientas de observabilidad (logging, métricas, trazas) utilizan actualmente para monitorizar el sistema en producción?
- ¿Cómo gestionan actualmente la consistencia de datos entre servicios? ¿Usan algún patrón como Saga o eventos de dominio?
- ¿Cuál es el mayor reto de arquitectura al que se han enfrentado en el último año?
- ¿Qué proceso siguen para tomar y documentar decisiones de arquitectura (ADRs, RFCs, etc.)?
- ¿Cómo es el proceso de despliegue y cuál es la frecuencia de entrega a producción?